

Disaster Management Using Real-Time Web Data, Chatbot Assistance, and VR Simulation

Kartik Chaudhary, Avaniash Kumar, Ashutosh Mishra, Animesh Kumar Singh,
Bhuvnesh Malik, and Umang Kant

kartikchaudharyy99@gmail.com, kavanishhere@gmail.com,
ashutoshbins@gmail.com, tronuprisingclg@gmail.com,
bhuvneshmaalik@gmail.com, umang.kant@kiet.edu

Department of CSE (AI&ML)
Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR,
Uttar Pradesh, India

Abstract. The need for rapid access to trustworthy information has become essential because disasters are occurring more frequently and their impact has increased. Our disaster management system which uses real-time data and ML technology to improve both preparedness and response operations. The system provides a mobile application which broadcasts alerts enables users to communicate without internet access and delivers information updates. The VR experience uses Unity to create a training platform which simulates various disaster scenarios.

Keywords: Disaster Management, Web Scraping, NLP Classification, SVM, Naive Bayes, Smart India Hackathon 2024 (SIH1687) project, Chatbot, LLM, Flutter, VR Simulation,

Organization of the Paper

- Introduction
- Problem Statement and Scope
- Review of Literature
- Our Approach
- Results and System Evaluation
- Case Studies of Past Disasters
- Future Scope
- Conclusion

1 Introduction

The researchers developed a complete disaster management system which consists of The system uses Python to collect live web data. The system uses natural language processing (NLP) to categorize information through support vector machine (SVM) and Naive Bayes model. The Unity-based system enables users to

experience disaster training through Virtual Reality disaster simulations. The last 20 years have witnessed a sharp increase in disaster frequency and severity which resulted in 7300 major disasters between 2000 and 2019 that killed 123 million people and affected 4 billion people.[1] The commonality between disaster events shows that scientific and system advancements have improved prediction and preparation capabilities yet people enter critical situations without trustworthy information.[8] The spread of falsehoods together with contradictory communication and disaster knowledge gaps is increasing dangers in areas where people live together and lack basic necessities such as resources in India.[9]

Modern disaster work has to do more than just provide isolated data or reactive alerts. It needs to work in real time, send alerts in different ways, and learn from experience to help whole communities with all parts of the disaster cycle.[6]

There are many technical challenges to making this happen: 1. information from disasters comes from many different sites and formats; 2. we need to sort fact from fiction, find who to trust; 3. we need to send early alerts in seconds or minutes, not hours or days, to save lives and understand what is happening[9] [7] . To meet these needs, I have built a disaster system that joins together: Python-based real-time web scraping, NLP classifiers based on SVM and Naive Bayes, a smart chatbot that mixes rules with large language models for better user talks, a mobile app built with Flutter that works without a net but alerts you based on your location, and Unity VR simulations that teach about disaster events by letting users experience them as if they were in the disaster cycle. This multi-layer plan will help communities be ready, make information better, and get more lives saved.

2 Problem Statement and Scope

Even though the world has come so far in how we can forecast these problems and how we can respond to them communities are still faced with a lot of problems trying to get the right and accurate information to act on in time as its happening. The lack of a centralized platform to consolidate, verify and distribute accurate and verified information about disasters is a critical flaw that can be exploited.. Another major concern is the lack of real time automated verification of disaster related content, social media and other streams contain useful information but can also be rife with rumours or false alarms that just create confusion and panic or worse[8]. There needs to be machine learning based filtering and categorization in order to distinguish between high priority alerts and low priority or irrelevant information[9] [7].

Also disaster management tools rarely feature interactive or immersive education tools, which would help ensure the public is prepared for disasters and are aware of the safe-response procedures that are necessary during emergencies. [6] [3].

The scope of this work is bounded by the goal of developing a comprehensive, multi-modal platform that improves disaster preparedness and real-time

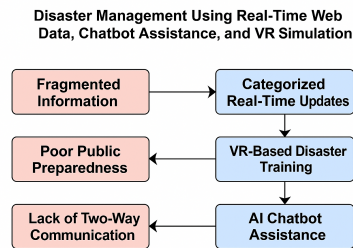


Fig. 1. Illustration of challenges in current disaster information systems.

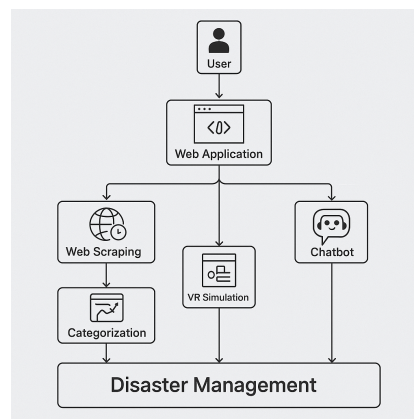


Fig. 2. System Workflow Representation of the Proposed Model

response. The proposed system focuses on: (1) automated acquisition and classification of disaster-related information, (2) accessible delivery of alerts and safety resources through a mobile application with offline capability, (3) an AI-assisted chatbot for interactive guidance, and (4) a VR module for awareness and training. the system does not do satellite based forecasting or large sensor networks for live rescue coordination it does not replace or tie into the real world it aims to support our existing institutions to make our communities more aware prepared and reliable.

2.1 Threat and Challenge Landscape

In addition to the poor communication, many tech and process threats can limit the use of modern disaster-management systems. One threat is fake news and fake alerts on social media[9]. When there is a disaster, posts that are fake can spread faster than the news reports that are true. It becomes more important to filter, classify, and check the reports as soon as possible. This in turn, calls for robust NLP models that are capable of handling poorly structured, differently spoken, and context-related content efficiently. [7]

Lag in data pipelines is posed as another threat. The situation in which it takes a considerable amount of time for data to be scraped, worked on, or sent out is definitely a problem, especially for those cases that are rapidly evolving, such as floods or earthquakes. In order to get rid of this problem, it is necessary to have an excellent design and small models that can work at a high speed.

Also, watch out for platform problems. Some social platforms, like X, are limiting how much you can scrape or view. [5] These limits show why offline options, using less data, and very small models are important if you want everyone to use new tech without issues.[6]

2.2 Ethical Considerations

Definitely if a system with AI that is involved in emergencies will be used, then the data safety should be prioritized first. Posts, GPS and the way people use a site can all be sources of private information. The system has to secure all that also anonymize user IDs, and comply with data regulations in order to prevent hacking or misuse.[13] Some updates may be overlooked. To solve the problem, the system should be vigilant about bias and it should re-train itself with data from everywhere so that things remain fair for everyone.[8] Getting the balance right between where alerts come from and where they go is key, because people need to get alerts from trusted sources that are accurate and on time.[4]

3 Review of Literature

Intelligent Disaster Response is becoming more aware as we find more efficient ways to handle crisis information. New tools are using information, data and technology to help people share information and figure out what to do in case

of a disaster.[2]. Since online information is rapidly changing and can be quite disorganized it is hard to decide what is true confirm details and give updates quickly.[7]

A lot of studies has shown that social media like Twitter and Facebook can share quick updates during disasters. Techniques using SVM, Naive Bayes and Transformer models are good at sorting short texts, but fake stuff, different languages, and suspect sources create problems. [8]. Research on disaster chatbots says they can help send info fast, list first-aid steps, and chat back and forth during a crisis. But many systems only use set answers and don't work well when disaster situations change quickly.[9]

At present, language models provide versatile assistance and have a better understanding of the situation. Besides that, Virtual Reality is progressively being employed for teaching and training in the occurrence of the disaster. VR assists people in concentrating, recalling things, and getting prepared by presenting them with the dangerous situations of floods, earthquakes, and fires.[4] [10]

All in all, many studies show one key problem: though searching social media, designing chatbots, and training with VR can be made better, these systems often are separate. No full disaster tool that takes info from live web data, sorts it automatically, talks back and forth, and gives users VR training in one simple tool. This study makes new disaster tool that mix these tools in one easy-to-use system, using NLP tools, AI help, phone alerts, and VR training. [6] [11]

4 Our Approach

The system is designed to look simple with blocky layers. It is meant to get, sort, share, and show disaster data fast. There are five main parts in the system: a Python script engine to get data, a machine learning step to sort text, a Node.js server to manage programs, a Flutter app for phones to show updates fast, and a Unity VR tool for disaster practice. Each part can work at its own but works well with REST APIs, making it is easy to be updated.[5]

4.1 System Architecture Overview

The system uses a modular service based design with five main parts, those are - a Node.js API server, small Python app for NLP driven text sorting, MongoDB database for data storage, a Flutter app for real time alerts and offline use, and at last a Unity powered VR disaster training tool. These parts connect via small REST APIs to let them run individually, scale easily, and share data quickly. The different sub chapters describe the parts of the system in detail:

- **4.2 Data Acquisition Pipeline** – Python scripts to automate data gathering, cleaning, and ranking of newly occurred disasters.
- **4.3 NLP Classification Engine** – SvM, Naive Bayes classifiers that identify the types of alerts and recognize the presence of false data.

- **4.4 AI Chatbot Architecture** – combination of rules and LLM for flexible dialog.
- **4.5 Flutter Mobile Application** – functioning locally, giving you alerts dependent on your location, and enabling you to communicate with others.
- **4.6 VR Simulation Module** – Unity to construct the 3D environment for disaster drills.

4.2 Data Acquisition Pipeline

The data collection system is getting the news about disasters that are happening from social media and the news that is already known by using a Python tool. Static web pages are read with Requests, while JavaScript-loaded pages are handled by Selenium. The system is operating on a cron schedule which is regularly updating the data like text, time, images, and links to the original site.

In order to focus on the most valuable information, a relevance score R is given to each scraped item which is determined by the keyword intensity (K), time relevance (T), and source credibility (S):

$$R = \alpha K + \beta T + \gamma S$$

where α , β , and γ are empirically tuned weights. Posts with a score above a set limit get sent to the NLP classification system (Section 4.3). This small process makes sure that the updates are received on time and only the important information is gathered.

4.3 NLP-Based Classification Engine

The classifier tool gets each post you pull out and does a common NLP step: split it up into words, make all lower case, remove common words with little meaning, and cut words down to their root form.

$$x = f(\text{tokenize, lower, remove stopwords, stem})$$

The text vector is then run through TFIDF. It then goes to two trained models: an SVM classifier that can sort into more than one group and a Multinomial Naive Bayes model made for short, light text. Its predicted category $\hat{y}$ is computed as:

$$\hat{y} = \arg \max_{c \in C} P(c | x)$$

where C includes five disaster-related classes: *Alerts*, *Government Notices*, *Rescue Updates*, *Public Advisories*, and *Casualty Reports*. Using SVM and Naive Bayes together gives you a good mix of correct answers and quick results so you can tell the different big parts of fresh data as it rolls in.[7][15]

4.4 AI Chatbot Architecture

The chatbot system has a mixed design of rule-based intent classifier and large language model fallback used for difficult or ambiguous questions.[4] [12] When a user asks q , the system checks for matching keywords or predefined intent templates; if it cannot identify the intent with high confidence, it sends q to a lightweight API powered by a language model that generates responses on the fly. The intent determination involves choosing from:

$$I = \arg \max P(I_k | q)$$

where I_k is the most likely class of intent. flow of conversation is kept with a very simple FSM, able to keep multi-turn dialogs going for all the pre disaster, during disaster and post disaster circumstances. this is very useful for simple queries, but it is also more flexible for anything else the user wants to do.

4.5 Flutter Mobile Application

The app was made with Flutter and is the main screen for learning about disaster updates. It is equipped with a cache system, therefore it can function offline and utilizes SQLite, so that you can access key information even if you don't have a signal. Real-time alerts are delivered through FCM. Using the Google Maps API, you can receive warnings about geolocation or look at the hazards. With Bloc/Riverpod, the app's behavior is made to be predictable and simple to change in the user interface.

the REST calls to the Node.js backend are minimal and swift, also a low-bandwidth mode is included so that the app remains responsive and is more convenient to use with limited internet connection.

4.6 VR Simulation Module

The virtual reality segment employs the Unity engine to create an immersive experience for disaster training.[14] Unity's SceneLoader, which is the means of changing the user interface from one to another, is used by users to navigate through different disasters such as floods, quakes, and fires in brief sessions. Earthquake scenes use RigidBody physics to shake the ground, flood scenes use easy NavierStokes math to make water look real. Custom code in C# makes the user play their part and decide what to do next in our Flutter app.[6]

In this way we can send data both ways and then people can sign in and pick scenes from both.

5 Results and System Evaluation

Evaluate the system across its key parts to see how well it classifies, how fast it is, how quick it responds, and how good it is for users. Run tests on a set of posts

taken from news sites and social media sites over two weeks[7]. The NLP classifier worked well most of the time, with the SVM method being more accurate in classifying many types of classes, while the Multinomial Naive Bayes method was faster when working with short text inputs. The scraping tool was very fast even with it running all the time on a schedule, and the mobile app worked reliably whether connected or not. The system results are written down in Table 1. This test shows that the full system can take and send disaster updates nearly

Table 1. System Performance Evaluation

Component	Performance Metric
Scraping Pipeline	1.2 s avg. extraction latency
SVM Classifier	82% accuracy
Naive Bayes Classifier	76% accuracy (faster inference)
Chatbot Response Time	0.5 s avg.
Flutter App Offline Load	190 ms
VR Module Frame Rate	55–60 FPS

as soon as they happen. The conversation-agent in hybrid-mode was efficient in handling both types of questions i.e., filled as well as empty, and the Unity VR component was running smoothly without any lag and was nice for training purposes. In sum, this experiment demonstrates the innovative system as a vehicle to propagate information in a swift manner, sustain the engagement level of the users, and foster the development of a robust preparedness strategy.[11]

6 Case Studies of Past Disasters

Let’s look at three disasters to see how this system could help. Each disaster had trouble with communication, keeping people informed, and getting useful info out fast.

Kerala Floods (2018) The 2018 Kerala floods displaced more than a million people and the communication of dam releases, rescue points, and shelters was very problematic. In the flooded situation, the use of social media was very high; however, there were many false and mixed updates. With the new technology, people should have been able to see everything that was happening live; also, the system may have automatically differentiated the news by using computer vision in order to eliminate false news, and in that way, targeted notifications could have been sent to the correct people. Moreover, discussing through VR about how to behave during floods might have helped people grasp the situation as well as the evacuation process.

Nepal Earthquake (2015) That huge 7.8 quake cut off Nepal leaving tons without any word on what to do after or where to find help. Now if they had the

app, it would have been saved on their phones and worked even with cruddy or no internet. Folks could still get safety tips. The bot could've been a lifesaver, showing first responders and locals first aid, who to call and custom tips for their area.[8].

Cyclone Amphan (2020) Cyclone Amphan caused a lot of harm in India and Bangladesh areas that were largely rural and where people did not have enough time to listen to a warning and leave. The bot's data pipeline might have combined the warnings from weather bureaus and town halls into one clear and simple notification. The interactive map could have shown people where it would have been safe to go, and the VR game could have prepared them for the future by teaching them what to do when a cyclone was approaching.[9] These stories how how a system, giving information right away and is easy to use, could really help keep people out of danger, squash fake news, and be a big help when disaster strike.

Figure 4 compares how often disasters happened and how many people died between 1900 and 2020.

7 Future Scope

The system design is adaptable and can be developed in numerous ways to make it more efficient and practical in everyday life. The main points of the future roadmap are:

- **Integration with NDMA and Government APIs:** Never rely on intermediary services for alerts, safety procedures, and threat maps, take them directly from the source for more security.
- **Predictive Modeling:** Use LSTM as a supplementary tool for weather prediction by forecasted flood height, storm power, or aftershock risk .[2]
- **Multilingual Support:** Incorporating chatbot and mobile words in other languages would facilitate communication with people in remote areas where languages differ or in villages. [2]
- **Crowdsourced Reporting:** User-generated hazard reports with trust scores could also be a way to prompt local first responders to arrive on time and create a rapid response scene account.
- **Scalable Cloud Deployment:** Moving to smaller servers on AWS or GCP means apps can grow without restarting.

These changes will help make the system faster, stronger, and better at handling disasters.

8 Conclusion

This paper introduces a disaster plan equipped with all the latest tools you might need. It is capable of receiving data in real-time, categorizes it using

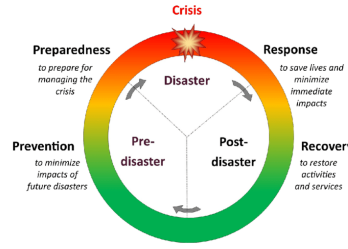


Fig. 3. Disaster Management Cycle: the phases of mitigation, preparedness, response and recovery

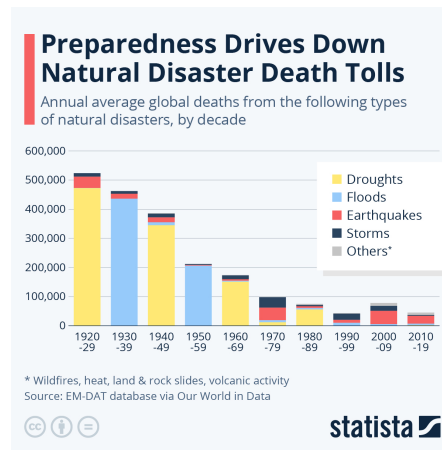


Fig. 4. Global Disaster Trends: Number of Events and Deaths b/w 1900–2020

an NLP classifier, employs a talking bot that combines rules and a LLM, and operates a VR training program that gives you the sensation of being there. This plan addresses the major issues of the modern disaster world - the data mess, fake news, and rapid spread of false information - by using a configuration that can add new features and become big. Testing results are a testament to its performance and efficiency for tech people with the SVM scoring 82, the Naive Bayes being quick, the scraping tool being low delay, and the Flutter app being very fast and able to run even if offline or on a slow network. The hybrid chatbot users the system better by giving safe and correct answers from rules or flexible answers from a LLM, and the Unity VR feature offers high quality, smooth, and real-feeling training.[6] [4]

This project is not just about the technology it shows why saving lives is important. Disasters don't have a "coming soon" period, they happen so suddenly. We have to prepare ourselves in advance for them. Our design also means it can adapt to new data, is not restricted to language and can also help government rescue agencies.

References

1. K. Sapountzaki, "Risk Mitigation, Vulnerability Management, and Resilience under Disasters," *Sustainability*, vol. 14, no. 6, pp. 1–8, 2022. [Online]. Available: <https://doi.org/10.3390/su14063589>
2. H. L. Tay, R. Banomyong, P. Varadejsatitwong, and P. Julagasigorn, "Mitigating Risks in the Disaster Management Cycle," *Advances in Civil Engineering*, vol. 2022, Article ID 7454760, 2022. [Online]. Available: <https://doi.org/10.1155/2022/7454760>
3. Anonymous, "Poverty and Disaster Resilience," *Independent Research Paper*, 2020.
4. A. Rossi, "The ERMES Chatbot: A Conversational Communication Tool for Improved Emergency Management and Disaster Risk Reduction," *International Journal of Disaster Risk Reduction*, vol. 112, 2024. [Online]. Available: <https://doi.org/10.1016/j.ijdrr.2024.104792>
5. K. Kopanov, V. Varbanov, and T. Atanasova, "Leveraging Social Media Data and Artificial Intelligence for Improving Earthquake Response Efforts," *arXiv preprint arXiv:2501.14767*, 2024. [Online]. Available: <https://arxiv.org/abs/2501.14767>
6. S. Khanal et al., "Virtual and Augmented Reality in Disaster Management Technology: A Literature Review of the Past 11 Years," *Frontiers in Virtual Reality*, vol. 3, 2022. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/frvir.2022.843195/full>
7. F. Alam, F. Ofli, and M. Imran, "CrisisMMD: Multimodal Twitter Datasets from Natural Disasters," *arXiv preprint arXiv:1805.00713*, 2018. [Online]. Available: <https://arxiv.org/abs/1805.00713>
8. T. H. Nazer et al., "Intelligent Disaster Response via Social Media Analysis: A Survey," *arXiv preprint arXiv:1709.02426*, 2017. [Online]. Available: <https://arxiv.org/abs/1709.02426>
9. Z. S. Dong, "Social Media Information Sharing for Natural Disaster Response," *arXiv preprint arXiv:2005.07019*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.07019>
10. O. J. M. Peña Cáceres et al., "Chatbot System for Data Management: A Case Study of Disaster-related Data," *ResearchGate*, 2024. [Online]. Available: <https://www.researchgate.net/publication/334127275>
11. L. Dupont et al., "Collaborative Virtual Reality Environment in Disaster Medicine: Moving from Single Player to Multiple Learners," *BMC Medical Education*, vol. 24, no. 422, 2024. [Online]. Available: <https://doi.org/10.1186/s12909-024-05429-8>
12. U. Kant, P. Dahiya, R. Priyadarshi, and M. Kumar, "Generative AI in Communication System Simulation and Modeling," in *Advances in Communication Systems Modeling*, 2025. [Online]. Available: <https://doi.org/10.2174/97898153246931250401>
13. P. Dahiya and U. Kant, "Multi-Factor Authentication Methods for Intelligent Systems," in *Intelligent Systems Security*, CRC Press, 2023. [Online]. Available: <https://doi.org/10.1201/9781032630748-7>
14. A. Mahmoud, "Sustainable Virtual Worlds: Implementing Renewable Energy Solutions," in *Green Metaverse: Fuzzy Logic Approaches to Sustainable Virtual Worlds*. Cham: Springer Nature Switzerland, 2026, pp. 111–133.
15. S. Sharma, "Performance Evaluation of the Deep Learning Based Convolutional Neural Network Approach for the Recognition of Chest X-ray Images," *Frontiers in Oncology*, vol. 12, p. 932496, 2022.